

OOP

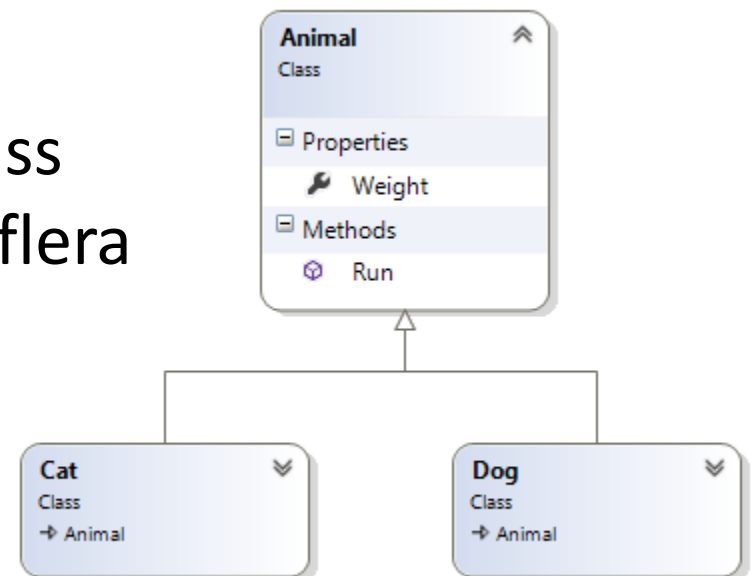
Arv – Koncept

Genom arv kan vi skapa klasser som får tillgång till egenskaper och metoder från en annan klass.

Den ärvda klassen kallas *basklass*.

Den ärvande klassen kallas *subklass*.

Vi utgår från en befintlig basklass och specialiserar den i en eller flera subklasser.



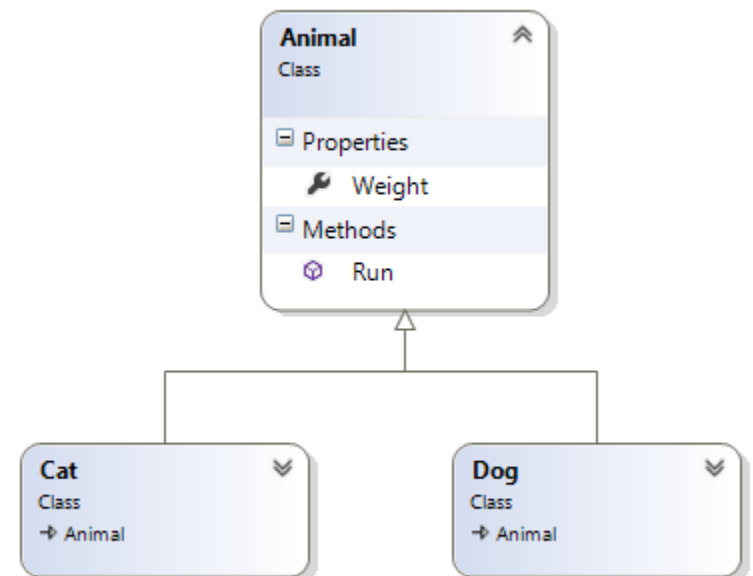
Arv – Relation mellan klasser

En klassrelation ska kunna beskrivas som att *subklass* är ett slags *basklass*. På engelska säger man **is a**.

Ex:

A Cat **is an** Animal

A SportsCar **is a** Car



Arv – Kod

Subklassen anger dess basklass med ett kolon.

```
// Subclass  
class Cat : Animal  
{  
}
```

Arv – Konstruktor

Konstruktorn är en metod som anropas när klassen instansieras.

Konstruktorn anropar i sin tur basklassens konstruktor, explicit eller implicit.

```
class Animal
{
    public Animal(int weight)
    {
        Weight = weight;
    }

    public int Weight { get; set; }
}
```

```
class Cat : Animal
{
    public Cat(int weight):
        base(weight)
    {
    }
}
```

Arv – Single Inheritance

C# tillämpar *single inheritance*, dvs. en klass kan bara ha en basklass.

Alla klasser ärver direkt eller indirekt från basklassen *Object*, som bl.a. erbjuder metoden *ToString*.

Polymorfism

Subklasser specialiserar basklassens beteende, utan att ta i bort egenskaper eller metoder. En subklass kan ändra eller utöka dessa.

Regel 1: Subklasser kan hanteras som instanser av deras basklasser

Regel 2: Då vi nyttjar regel 1 så används automatiskt den mest specialiserade metoden eller egenskapen.

Polymorfism

Regel 1: Subklasser kan hanteras som instanser av deras basklasser

```
var cars = new List<Car>();  
cars.Add(new Car());  
// Regel #1 - sportbilar kan hanteras som bilar  
cars.Add(new SportsCar());
```


Polymorfism

Regel 2: Då vi nyttjar regel 1 så används automatiskt den mest specialiserade metoden eller egenskapen.

```
var cars = new List<Car>();  
cars.Add(new Car());  
// Regel #1 - sportbilar kan hanteras som bilar  
cars.Add(new SportsCar());  
  
foreach (var car in cars)  
{  
    // Regel #2  
    // Kommer anropa Car.Start() för bilar  
    // Kommer anropa SportsCar.Start() för sportbilar  
    car.Start();  
}
```

Polymorfism – Virtuella metoder

Ett av verktygen för att specialisera subklassen.

Genom att markera en metod **virtual** tillåter man att en subklass ersätter den med en egen, sk **override**.

```
// Basklass
class Car
{
    public int Speed { get; set; }

    public virtual void Start()
    {
        Speed = 50;
    }
}

// Subklass
class SportsCar : Car
{
    override public void Start()
    {
        Speed = 100;
    }
}
```

Polymorfism – Abstrakta metoder

Abstrakta metoder saknar implementation.

En klass som innehåller en abstrakt metod måste själv vara abstract, och får alltså inte instansieras.

Genom att markera en metod **abstract** tvingar man att en subklass ersätter den med en egen.

```
// Basklass
abstract class Car
{
    public int Speed { get; set; }

    public abstract void Start();
}
```

Static

Nyckelordet **static** kan anges på klasser och/eller dess metoder, egenskaper och fält.

Statiska medlemmar delas av *samtliga* instanser och kan sägas höra till klassen snarare än instanserna.

En klass som är markerad som **static** får bara innehålla statiska medlemmar och kan följaktligen inte instanseras.

Static – Exempel

Vi har redan använt några statiska metoder:

Program.Main()

Console.WriteLine()

Console.ReadLine()

String.IsNullOrEmpty()

Static – Anrop

Anrop av statiska klasser görs utifrån klassen genom *klassnamnet*.

```
// PersonCount anropas på klassen  
Console.WriteLine(Person.PersonCount);
```

```
// Name anropas på instansen  
var person = new Person();  
Console.WriteLine(person.Name);
```

Static – Användning

Statiska klasser är ofta sk utility-klasser, verktygslådor eller hjälpklasser.

```
static class StringUtilsils
{
    public static bool IsNumeric(string value)
    {
        int i;
        return int.TryParse(value, out i);
    }
}
```

```
Console.WriteLine(StringUtilsils.IsNumeric("Test")); // False
Console.WriteLine(StringUtilsils.IsNumeric("10")); // True
```

Static – Konstruktorer

Första gången en klass instansieras eller en statisk medlem anropas kommer klassens statiska konstruktor anropas. Detta är också sista gången det görs.

Har ingen access-modifierare (är alltid publik).

```
class Person
{
    static Person()
    {
    }
}
```